Problem1

May 3, 2026

0.1 2.1 Cart-pole swing-up with limited actuation

(a)

The convex approximation (LOCP) around the current iterate $(s^{(i)}, u^{(i)})$ is

$$\begin{aligned}
\text{minimize}_{s,\,u} \quad & (s_N - s_{\text{goal}})^\top P\,(s_N - s_{\text{goal}}) \\
& + \sum_{k=0}^{N-1} \left[(s_k - s_{\text{goal}})^\top Q\,(s_k - s_{\text{goal}}) + u_k^\top R\,u_k\right] \\
\text{subject to} \quad & s_0 = \bar{s}_0 \\
& u_k \in \mathcal{U}, \quad \forall k \in \{0, \dots, N-1\} \\
& \|u_k - u_k^{(i)}\|_\infty \le \rho, \quad \forall k \in \{0, \dots, N-1\} \\
& \|s_k - s_k^{(i)}\|_\infty \le \rho, \quad \forall k \in \{0, \dots, N\} \\
& s_{k+1} = A_k s_k + B_k u_k + c_k, \quad \forall k \in \{0, \dots, N-1\}
\end{aligned}$$

where the terminal constraint $s_N = s_{\text{goal}}$ is replaced by the terminal cost $(s_N - s_{\text{goal}})^\top P\,(s_N - s_{\text{goal}})$, and A_k, B_k, c_k come from the linearization as follow:

$$s_{k+1} = f_{\mathbf{d}}\big(s_k^{(i)}, u_k^{(i)}\big) + \frac{\partial f_{\mathbf{d}}}{\partial s}\big(s_k^{(i)}, u_k^{(i)}\big)\big(s_k - s_k^{(i)}\big) + \frac{\partial f_{\mathbf{d}}}{\partial u}\big(s_k^{(i)}, u_k^{(i)}\big)\big(u_k - u_k^{(i)}\big) = A_k s_k + B_k u_k + c_k$$

with

$$A_k = \frac{\partial f_{\mathbf{d}}}{\partial s}\big(s_k^{(i)}, u_k^{(i)}\big), \qquad B_k = \frac{\partial f_{\mathbf{d}}}{\partial u}\big(s_k^{(i)}, u_k^{(i)}\big), \qquad c_k = f_{\mathbf{d}}\big(s_k^{(i)}, u_k^{(i)}\big) - A_k s_k^{(i)} - B_k u_k^{(i)}$$

(b) Please see `cartpole_swingup_constrained.py`.

(c)

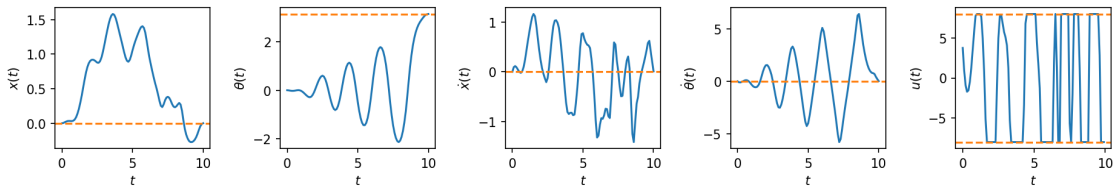
Please see `cartpole_swingup_constrained.py`.

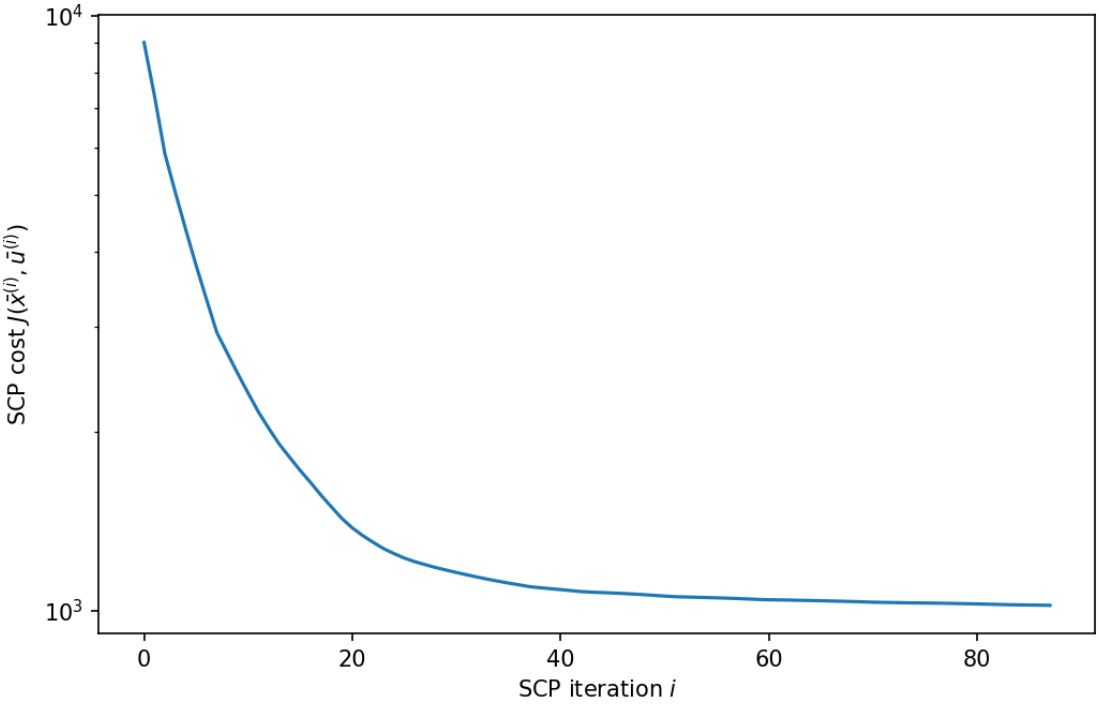
(d)

[1]:
```
%run cartpole_swingup_constrained.py
```

```
 88%|      | 88/100 [01:21<00:11,  1.08it/s, objective change=0.32265]
```

```
SCP converged after 88 iterations.
```





The pendulum just reaches s_{goal} at the end of the horizon but does not stay there. A natural scheme is to use SCP to generate the swing-up trajectory, then switch to a closed-loop LQR controller linearized around the upright equilibrium to keep the pendulum there indefinitely.

Problem2

May 3, 2026

0.1 2.2 Shortest path through a grid

(a)

Doing DP backward (B to A) we compute at each node the optimal cost-to-go (J^* in the figure) and store the optimal edge to follow, represented as a blue arrow. Finally, the optimal path from A to B is computed by always choosing the optimal edge and is highlighted in red. It corresponds to: **up, up, down, up, down, down**.

[1]:
```
from IPython.display import Image, display
display(Image(filename='/Users/erwinpoussi/Downloads/NBMetadataCache 2.png',␣
↪width=700))
```

ight and t tr
ecision to go node. Given
ow-dimensional state space, we will use dynamic prog
each node in the grid.

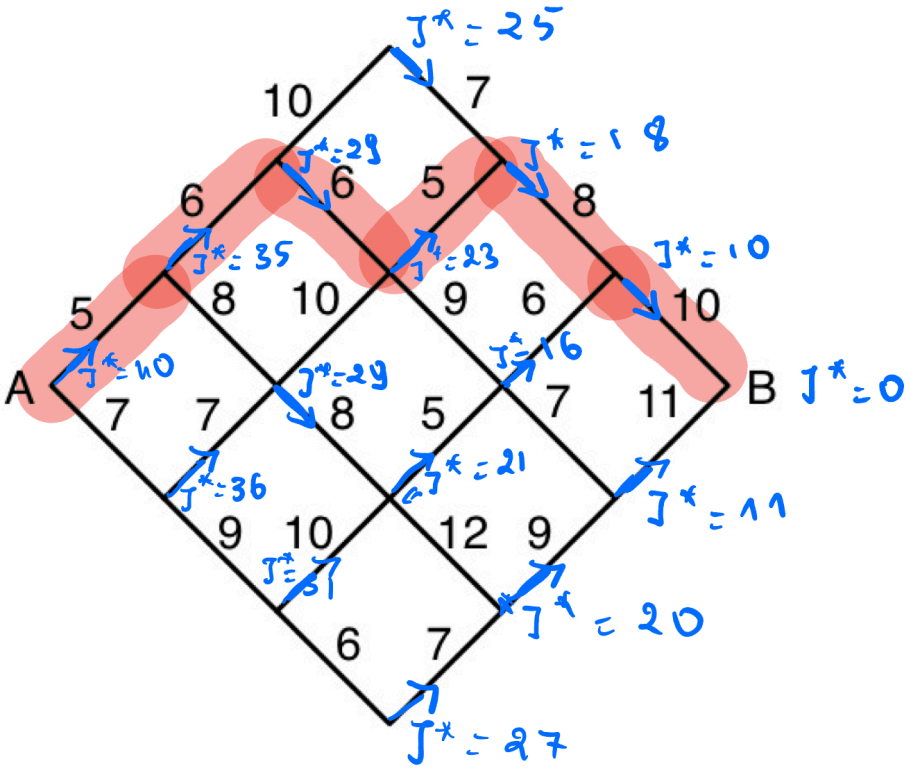


Figure 1: Shortest path problem on a grid.

amming (DP) to find the shortest path from A to B
lized version of the shortest path problem in Figure
h side. Find the number of computations required b
number of routes that such an algorithm would nee

(b)

Number of nodes. For a grid with n segments per side, the diamond has columns $0, 1, \ldots, 2n$,

with the number of nodes per column growing from 1 to $n+1$ then shrinking back to 1:

$$n_{\text{nodes}} = \sum_{k=0}^{n}(k+1) + \sum_{k=1}^{n} k = \frac{(n+1)(n+2)}{2} + \frac{n(n+1)}{2} = (n+1)^2.$$

For $n = 3$: $n_{\text{nodes}} = 16$.

Exhaustive search. Every path from A to B consists of $2n$ steps, each either up or down, and must end at B (same row as A), which requires exactly n ups and n downs. The number of such paths is therefore

$$\boxed{N_{\text{exhaustive}} = \binom{2n}{n} = \frac{(2n)!}{(n!)^2}}.$$

For $n = 3$: $\binom{6}{3} = 20$ routes.

Dynamic programming. The DP backward sweep evaluates the cost-to-go J^* once at every node except the terminal B. At each node the computation is a single min over at most 2 successors, so the total number of DP evaluations equals the number of non-terminal nodes:

$$\boxed{N_{\text{DP}} = (n+1)^2 - 1 = n^2 + 2n}.$$

For $n = 3$: $N_{\text{DP}} = 9 + 6 = 15$ evaluations.

Comparison. $\binom{2n}{n}$ grows exponentially ($\sim 4^n/\sqrt{\pi n}$) while $n^2 + 2n$ grows only polynomially, illustrating the considerable computational advantage of DP over exhaustive search.

problem3

May 8, 2026

0.1 2.3 Cart-pole balance

(a)

Starting from the continuous-time dynamics $\dot{s} = f(s, u)$, we apply **Euler integration** to get the discrete-time approximation:

$$s_{k+1} \approx s_k + \Delta t\, f(s_k, u_k).$$

We then **linearize** f around the equilibrium $(\bar{s}, \bar{u})$ via a first-order Taylor expansion:

$$f(s_k, u_k) \approx f(\bar{s}, \bar{u}) + \left.\frac{\partial f}{\partial s}\right|_{(\bar{s},\bar{u})} (s_k - \bar{s}) + \left.\frac{\partial f}{\partial u}\right|_{(\bar{s},\bar{u})} (u_k - \bar{u}).$$

Since $(\bar{s}, \bar{u})$ is an equilibrium, $f(\bar{s}, \bar{u}) = 0$, and $\bar{u} = 0$. Defining $\tilde{s}_k = s_k - \bar{s}$ and subtracting $\bar{s}$ from both sides:

$$\tilde{s}_{k+1} \approx \tilde{s}_k + \Delta t \left(\left.\frac{\partial f}{\partial s}\right|_{(\bar{s},\bar{u})} \tilde{s}_k + \left.\frac{\partial f}{\partial u}\right|_{(\bar{s},\bar{u})} u_k \right) = \underbrace{\left(I + \Delta t \left.\frac{\partial f}{\partial s}\right|_{(\bar{s},\bar{u})} \right)}_{A} \tilde{s}_k + \underbrace{\Delta t \left.\frac{\partial f}{\partial u}\right|_{(\bar{s},\bar{u})}}_{B} u_k.$$

Substituting the given Jacobians evaluated at $\bar{s} = (0, \pi, 0, 0)$, $\bar{u} = 0$:

$$A = I_4 + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c+m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \qquad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}.$$

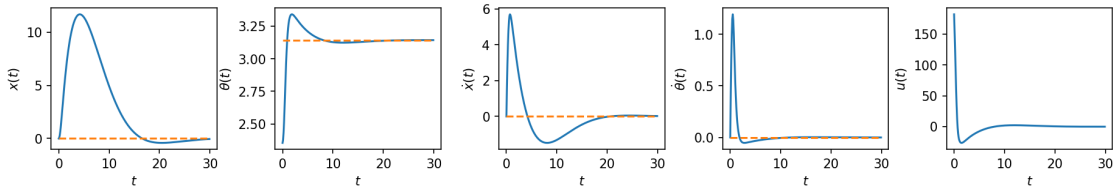
(b)

[1]:
```
import numpy as np
from cartpole_balance import compute_lti_matrices, ricatti_recursion

A, B = compute_lti_matrices()
Q, R = np.eye(4), np.eye(1)
K = ricatti_recursion(A, B, Q, R)
print("K_inf:\n", np.round(K, 2))
```

```
K: [[   0.7291397  -231.85419273    4.21967187  -68.24742822]]
K_inf:
 [[   0.73 -231.85    4.22  -68.25]]
```

(c)

[2]:
```
from cartpole_balance import simulate, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
u_ref = np.array([0.0])
s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
```



[3]:
```
from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

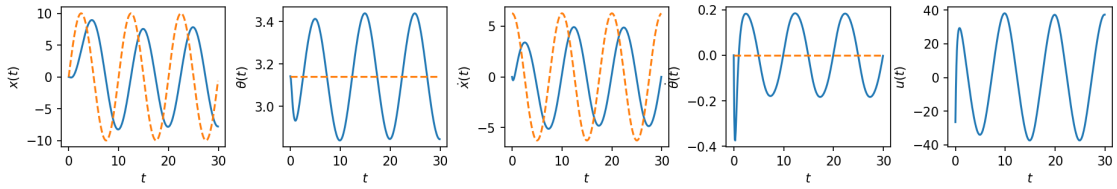
[3]:
```
<IPython.core.display.HTML object>
```

(d)

(i) The Jacobians $\frac{\partial f}{\partial s}$ and $\frac{\partial f}{\partial u}$ do not depend on the cart position x — only on θ, $\dot{\theta}$, and u. Since the reference trajectory always maintains $\bar{\theta}(t) = \pi$, $\dot{\bar{\theta}}(t) = 0$, and $\bar{u} = 0$, the linearization point is identical at every time step regardless of the time-varying $\bar{x}(t)$. Therefore A and B are constant and do not need to be recomputed.

(iii) The desired trajectory $\bar{x}(t) = a\sin(2\pi t/T)$ requires the cart to accelerate ($\ddot{\bar{x}} \neq 0$), which physically means a nonzero reference force $\bar{u}(t) \neq 0$ is needed to drive it. The LQR was designed assuming $\bar{u} = 0$, so there is no feedforward term to generate this force. As a result the controller cannot simultaneously track $\bar{x}(t)$ and keep $\theta = \pi$: the cart acceleration perturbs the pendulum, causing θ and $\dot{\theta}$ to oscillate. Increasing Q tightens regulation around the reference but does not fix the missing feedforward, so θ and $\dot{\theta}$ keep oscillating.

[4]:
```
from cartpole_balance import simulate, reference, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([reference(ti) for ti in t])
u_ref = np.array([0.0])
s0 = np.array([0.0, np.pi, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
```



[5]:
```
from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

[5]:
```
<IPython.core.display.HTML object>
```

[ ]:
```

```

problem4

May 3, 2026

0.1 2.4 Cart-pole swing-up

(a) Please see `cartpole_swingup.py`.

(b)

Using a second order Taylor expansion of the stage cost $c(\tilde{s}_k, \tilde{u}_k)$ around $(\bar{s}_k, \bar{u}_k)$:

$$c(\tilde{s}_k, \tilde{u}_k) = c(\bar{s}_k, \bar{u}_k) + \underbrace{\left(\left.\frac{\partial c}{\partial s}\right|_{(\bar{s}_k, \bar{u}_k)} \right)^\top}_{=q_k} \tilde{s}_k + \underbrace{\left(\left.\frac{\partial c}{\partial u}\right|_{(\bar{s}_k, \bar{u}_k)} \right)^\top}_{=r_k} \tilde{u}_k + \frac{1}{2} \tilde{s}_k^\top \underbrace{\left.\frac{\partial^2 c}{\partial s^2}\right|_{(\bar{s}_k, \bar{u}_k)}}_{=Q} \tilde{s}_k + \frac{1}{2} \tilde{u}_k^\top \underbrace{\left.\frac{\partial^2 c}{\partial u^2}\right|_{(\bar{s}_k, \bar{u}_k)}}_{=R} \tilde{u}_k$$

hence $\boxed{q_k = Q(\bar{s}_k - s_{\text{goal}})}$ and $\boxed{r_k = R\bar{u}_k}$.

Same for the terminal cost $c_N(\tilde{s}_N)$:

$$c_N(\tilde{s}_N) = c_N(\bar{s}_N) + \underbrace{\left(\left.\frac{\partial c_N}{\partial s}\right|_{\bar{s}_N} \right)^\top}_{=q_N} \tilde{s}_N + \frac{1}{2} \tilde{s}_N^\top \underbrace{\left.\frac{\partial^2 c_N}{\partial s^2}\right|_{\bar{s}_N}}_{=Q_N} \tilde{s}_N$$

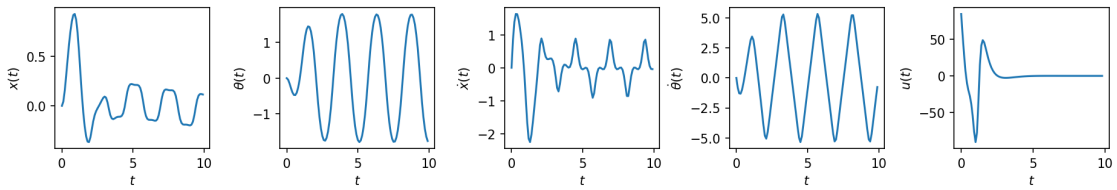
hence $\boxed{q_N = Q_N(\bar{s}_N - s_{\text{goal}})}$.

(c) Please see `cartpole_swingup.py`.

(d) Please see `cartpole_swingup.py`.

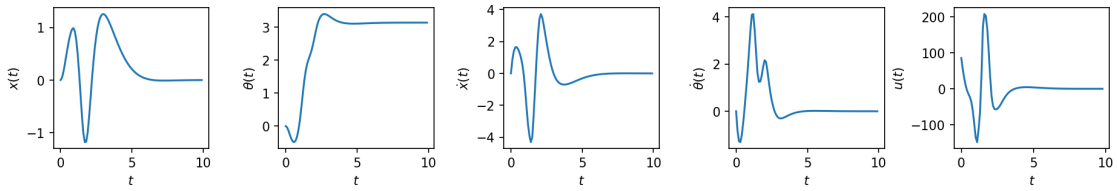
[1]:
```
from IPython.display import Image, display
print("Open-loop:")
display(Image(filename='cartpole_swingup_ol.png'))
```

```
Open-loop:
```



[2]:
```
print("Closed-loop:")
display(Image(filename='cartpole_swingup_cl.png'))
```

```
Closed-loop:
```



While the open-loop is consistently oscillating, the closed-loop is progressively damped towards convergence as expected.

Problem5

May 3, 2026

0.1 2.5 Machine maintenance

States: R (running), B (broken) at the start of each week.

Terminal: $J_5(R) = J_5(B) = 0$

Immediate expected net profits and transitions:

- R + Maintain: $0.6(100-20) + 0.4(0-20) = 40$, next state: R w.p. 0.6, B w.p. 0.4
- R + No maintain: $0.3(100) + 0.7(0) = 30$, next state: R w.p. 0.3, B w.p. 0.7
- B + Repair: $0.6(100-40) + 0.4(0-40) = 20$, next state: R w.p. 0.6, B w.p. 0.4
- B + Replace: $1(100-150) = -50$, next state: R w.p. 1

Week 4:

$J_4(R)$: Maintain $\to 40$, No maintain $\to 30 \Rightarrow J_4(R) = \max(40, 30) = \mathbf{40}$ (Maintain)

$J_4(B)$: Repair $\to 20$, Replace $\to -50 \Rightarrow J_4(B) = \max(20, -50) = \mathbf{20}$ (Repair)

Week 3:

$J_3(R)$: Maintain $\to 40 + 0.6(40) + 0.4(20) = 72$, No maintain $\to 30 + 0.3(40) + 0.7(20) = 56$ $\Rightarrow J_3(R) = \max(72, 56) = \mathbf{72}$ (Maintain)

$J_3(B)$: Repair $\to 20 + 0.6(40) + 0.4(20) = 52$, Replace $\to -50 + 40 = -10 \Rightarrow J_3(B) = \max(52, -10) = \mathbf{52}$ (Repair)

Week 2:

$J_2(R)$: Maintain $\to 40 + 0.6(72) + 0.4(52) = 104$, No maintain $\to 30 + 0.3(72) + 0.7(52) = 88$ $\Rightarrow J_2(R) = \max(104, 88) = \mathbf{104}$ (Maintain)

$J_2(B)$: Repair $\to 20 + 0.6(72) + 0.4(52) = 84$, Replace $\to -50 + 72 = 22 \Rightarrow J_2(B) = \max(84, 22) = \mathbf{84}$ (Repair)

Week 1 (new machine, guaranteed to run $\Rightarrow$ earns \$100, ends in state R, no maintain needed):

$$J_1 = 100 + J_2(R) = 100 + 104 = \boxed{204}$$

cartpole_balance.py

```python
"""
Solution code for the problem "Cart-pole balance".

Autonomous Systems Lab (ASL), Stanford University
"""

import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

from animations import animate_cartpole

# Constants
n = 4  # state dimension
m = 1  # control dimension
mp = 2.0  # pendulum mass
mc = 10.0  # cart mass
L = 1.0  # pendulum length
g = 9.81  # gravitational acceleration
dt = 0.1  # discretization time step
animate = False  # whether or not to animate results


def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
    """Compute the cart-pole state derivative

    Args:
        s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
        u (np.ndarray): The cartpole control: [F_x], shape (m,)

    Returns:
        np.ndarray: The state derivative, shape (n,)
    """
    x, θ, dx, dθ = s
    sinθ, cosθ = np.sin(θ), np.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = np.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


def reference(t: float) -> np.ndarray:
    """Compute the reference state (s_bar) at time t

    Args:
        t (float): Evaluation time

    Returns:
        np.ndarray: Reference state, shape (n,)
    """
    a = 10.0  # Amplitude
    T = 10.0  # Period

    # PART (d) ##################################################
    return np.array([
        a * np.sin(2 * np.pi * t / T),
        np.pi,
        a * (2 * np.pi / T) * np.cos(2 * np.pi * t / T),
        0.0,
    ])
    # END PART (d) ##############################################


def ricatti_recursion(
    A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
```

```python
) -> np.ndarray:
    """Compute the gain matrix K through Ricatti recursion

    Args:
        A (np.ndarray): Dynamics matrix, shape (n, n)
        B (np.ndarray): Controls matrix, shape (n, m)
        Q (np.ndarray): State cost matrix, shape (n, n)
        R (np.ndarray): Control cost matrix, shape (m, m)

    Returns:
        np.ndarray: Gain matrix K, shape (m, n)
    """
    eps = 1e-4  # Riccati recursion convergence tolerance
    max_iters = 1000  # Riccati recursion maximum number of iterations
    P_prev = np.zeros((n, n))  # initialization
    converged = False
    for i in range(max_iters):
        # PART (b) ###################################################
        K = -np.linalg.inv(R + B.T @ P_prev @ B) @ B.T @ P_prev @ A
        P = Q + A.T @ P_prev @ (A + B @ K)
        if np.max(np.abs(P - P_prev)) < eps:
            converged = True
            break
        P_prev = P
        # END PART (b) ###############################################
    if not converged:
        raise RuntimeError("Ricatti recursion did not converge!")
    print("K:", K)
    return K


def simulate(
    t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
) -> tuple[np.ndarray, np.ndarray]:
    """Simulate the cartpole

    Args:
        t (np.ndarray): Evaluation times, shape (num_timesteps,)
        s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
        u_ref (np.ndarray): Reference control u_bar, shape (m,)
        s0 (np.ndarray): Initial state, shape (n,)
        K (np.ndarray): Feedback gain matrix (Ricatti recursion result), shape (m, n)

    Returns:
        tuple[np.ndarray, np.ndarray]: Tuple of:
            np.ndarray: The state history, shape (num_timesteps, n)
            np.ndarray: The control history, shape (num_timesteps, m)
    """

    def cartpole_wrapper(s, t, u):
        """Helper function to get cartpole() into a form preferred by odeint, which expects t as the second arg"""
        return cartpole(s, u)

    # PART (c) ###################################################
    N = t.size - 1
    s = np.zeros((t.size, n))
    u = np.zeros((t.size, m))
    s[0] = s0
    for k in range(N):
        u[k] = K @ (s[k] - s_ref[k]) + u_ref
        s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], (u[k],))[1]
    u[N] = K @ (s[N] - s_ref[N]) + u_ref
    # END PART (c) ###############################################
    return s, u


def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
    """Compute the linearized dynamics matrices A and B of the LTI system

    Returns:
        tuple[np.ndarray, np.ndarray]: Tuple of:
            np.ndarray: The A (dynamics) matrix, shape (n, n)
            np.ndarray: The B (controls) matrix, shape (n, m)
    """
    # PART (a) ###################################################
    A = np.eye(n) + dt * np.array([
```

```python
        [0, 0, 1, 0],
        [0, 0, 0, 1],
        [0, mp * g / mc, 0, 0],
        [0, (mc + mp) * g / (mc * L), 0, 0],
    ])
    B = dt * np.array([[0], [0], [1 / mc], [1 / (mc * L)]])
    # END PART (a) ###############################################
    return A, B


def plot_state_and_control_history(
    s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
) -> None:
    """Helper function for cartpole visualization

    Args:
        s (np.ndarray): State history, shape (num_timesteps, n)
        u (np.ndarray): Control history, shape (num_timesteps, m)
        t (np.ndarray): Times, shape (num_timesteps,)
        s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
        name (str): Filename prefix for saving figures
    """
    fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
    plt.subplots_adjust(wspace=0.35)
    labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
    labels_u = (r"$u(t)$",)
    for i in range(n):
        axes[i].plot(t, s[:, i])
        axes[i].plot(t, s_ref[:, i], "--")
        axes[i].set_xlabel(r"$t$")
        axes[i].set_ylabel(labels_s[i])
    for i in range(m):
        axes[n + i].plot(t, u[:, i])
        axes[n + i].set_xlabel(r"$t$")
        axes[n + i].set_ylabel(labels_u[i])
    plt.savefig(f"{name}.png", bbox_inches="tight")
    plt.show()

    if animate:
        fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
        ani.save(f"{name}.mp4", writer="ffmpeg")
        plt.show()


def main():
    # Part A
    A, B = compute_lti_matrices()

    # Part B
    Q = np.eye(n)  # state cost matrix
    R = np.eye(m)  # control cost matrix
    K = ricatti_recursion(A, B, Q, R)

    # Part C
    t = np.arange(0.0, 30.0, 1 / 10)
    s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
    u_ref = np.array([0.0])
    s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
    s, u = simulate(t, s_ref, u_ref, s0, K)
    plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")

    # Part D
    # Note: t, u_ref unchanged from part c
    s_ref = np.array([reference(ti) for ti in t])
    s0 = np.array([0.0, np.pi, 0.0, 0.0])
    s, u = simulate(t, s_ref, u_ref, s0, K)
    plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")


if __name__ == "__main__":
    main()
```

cartpole_swingup.py

```python
"""
Starter code for the problem "Cart-pole swing-up".

Autonomous Systems Lab (ASL), Stanford University
"""

import time

from animations import animate_cartpole

import jax
import jax.numpy as jnp

import matplotlib.pyplot as plt

import numpy as np

from scipy.integrate import odeint


def linearize(f, s, u):
    """Linearize the function `f(s, u)` around `(s, u)`.

    Arguments
    ---------
    f : callable
        A nonlinear function with call signature `f(s, u)`.
    s : numpy.ndarray
        The state (1-D).
    u : numpy.ndarray
        The control input (1-D).

    Returns
    -------
    A : numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `s`.
    B : numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `u`.
    """
    # WRITE YOUR CODE BELOW ###################################################
    # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
    A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
    ###########################################################################
    return A, B


def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
    """Compute the iLQR set-point tracking solution.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
```

```
    s_goal : numpy.ndarray
        The goal state (1-D).
    N : int
        The time horizon of the LQR cost function.
    Q : numpy.ndarray
        The state cost matrix (2-D).
    R : numpy.ndarray
        The control cost matrix (2-D).
    QN : numpy.ndarray
        The terminal state cost matrix (2-D).
    eps : float, optional
        Termination threshold for iLQR.
    max_iters : int, optional
        Maximum number of iLQR iterations.

    Returns
    -------
    s_bar : numpy.ndarray
        A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u_bar : numpy.ndarray
        A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
        for `k = 0, 1, ..., N-1`
    Y : numpy.ndarray
        A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
        law at time step `k`, for `k = 0, 1, ..., N-1`
    y : numpy.ndarray
        A 2-D array where `y[k]` is the offset term of the iLQR control law
        at time step `k`, for `k = 0, 1, ..., N-1`
    """
    if max_iters <= 1:
        raise ValueError("Argument `max_iters` must be at least 1.")
    n = Q.shape[0]  # state dimension
    m = R.shape[0]  # control dimension

    # Initialize gains `Y` and offsets `y` for the policy
    Y = np.zeros((N, m, n))
    y = np.zeros((N, m))

    # Initialize the nominal trajectory `(s_bar, u_bar`), and the
    # deviations `(ds, du)`
    u_bar = np.zeros((N, m))
    s_bar = np.zeros((N + 1, n))
    s_bar[0] = s0
    for k in range(N):
        s_bar[k + 1] = f(s_bar[k], u_bar[k])
    ds = np.zeros((N + 1, n))
    du = np.zeros((N, m))

    # iLQR loop
    converged = False
    for _ in range(max_iters):
        # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
        A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
        A, B = np.array(A), np.array(B)

        # PART (c) ###################################################
        # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.

        # Backward pass: Riccati recursion for P (value Hessian), p (value gradient),
```

```
        # gains Y[k] and offsets y[k]
        P = QN
        p = QN @ (s_bar[-1] - s_goal)
        for k in range(N - 1, -1, -1):
            inv = np.linalg.inv(R + B[k].T @ P @ B[k])
            Y[k] = -inv @ B[k].T @ P @ A[k]
            y[k] = -inv @ (R @ u_bar[k] + B[k].T @ p)
            p = Q @ (s_bar[k] - s_goal) + A[k].T @ (p + P @ B[k] @ y[k])
            P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])

        # Forward pass: apply policy on real dynamics
        s_new = np.zeros_like(s_bar)
        u_new = np.zeros_like(u_bar)
        s_new[0] = s0
        for k in range(N):
            du[k] = Y[k] @ (s_new[k] - s_bar[k]) + y[k]
            u_new[k] = u_bar[k] + du[k]
            s_new[k + 1] = f(s_new[k], u_new[k])
        s_bar = s_new
        u_bar = u_new

        ############################################################################

        if np.max(np.abs(du)) < eps:
            converged = True
            break
    if not converged:
        raise RuntimeError("iLQR did not converge!")
    return s_bar, u_bar, Y, y


def cartpole(s, u):
    """Compute the cart-pole state derivative."""
    mp = 2.0  # pendulum mass
    mc = 10.0  # cart mass
    L = 1.0  # pendulum length
    g = 9.81  # gravitational acceleration

    x, θ, dx, dθ = s
    sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = jnp.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


# Define constants
n = 4  # state dimension
m = 1  # control dimension
Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0]))  # state cost matrix
R = 1e-2 * np.eye(m)  # control cost matrix
QN = 1e2 * np.eye(n)  # terminal state cost matrix
s0 = np.array([0.0, 0.0, 0.0, 0.0])  # initial state
```

```python
s_goal = np.array([0.0, np.pi, 0.0, 0.0])  # goal state
T = 10.0  # simulation time
dt = 0.1  # sampling time
animate = False  # flag for animation
closed_loop = True  # flag for closed-loop control

# Initialize continuous-time and discretized dynamics
f = jax.jit(cartpole)
fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))

# Compute the iLQR solution with the discretized dynamics
print("Computing iLQR solution ... ", end="", flush=True)
start = time.time()
t = np.arange(0.0, T, dt)
N = t.size - 1
s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
print("done! ({:.2f} s)".format(time.time() - start), flush=True)

# Simulate on the true continuous-time system
print("Simulating ... ", end="", flush=True)
start = time.time()
s = np.zeros((N + 1, n))
u = np.zeros((N, m))
s[0] = s0
for k in range(N):
    # PART (d) ###############################################################
    # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
    # `u[k]`, depending on the Boolean flag `closed_loop`.
    if closed_loop:
        u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
    else:  # do open-loop control
        u[k] = u_bar[k]
    ##########################################################################
    s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
print("done! ({:.2f} s)".format(time.time() - start), flush=True)

# Plot
fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
plt.subplots_adjust(wspace=0.45)
labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
labels_u = (r"$u(t)$",)
for i in range(n):
    axes[i].plot(t, s[:, i])
    axes[i].set_xlabel(r"$t$")
    axes[i].set_ylabel(labels_s[i])
for i in range(m):
    axes[n + i].plot(t[:-1], u[:, i])
    axes[n + i].set_xlabel(r"$t$")
    axes[n + i].set_ylabel(labels_u[i])
if closed_loop:
    plt.savefig("cartpole_swingup_cl.png", bbox_inches="tight")
else:
    plt.savefig("cartpole_swingup_ol.png", bbox_inches="tight")
plt.show()

if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup.mp4", writer="ffmpeg")
    plt.show()
```

cartpole_swingup_constrained.py

```python
"""
Starter code for the problem "Cart-pole swing-up with limited actuation".

Autonomous Systems Lab (ASL), Stanford University
"""

from functools import partial

from animations import animate_cartpole

import cvxpy as cvx

import jax
import jax.numpy as jnp

import matplotlib.pyplot as plt

import numpy as np

from tqdm import tqdm


@partial(jax.jit, static_argnums=(0,))
@partial(jax.vmap, in_axes=(None, 0, 0))
def affinize(f, s, u):
    """Affinize the function `f(s, u)` around `(s, u)`.

    Arguments
    ---------
    f : callable
        A nonlinear function with call signature `f(s, u)`.
    s : numpy.ndarray
        The state (1-D).
    u : numpy.ndarray
        The control input (1-D).

    Returns
    -------
    A : jax.numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `s`.
    B : jax.numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `u`.
    c : jax.numpy.ndarray
        The offset term in the first-order Taylor expansion of `f` at `(s, u)`
        that sums all vector terms strictly dependent on the nominal point
        `(s, u)` alone.
    """
    # PART (b) #############################################################
    # INSTRUCTIONS: Use JAX to affinize `f` around `(s, u)` in two lines.
    A,B=jax.jacobian(f, argnums=(0,1))(s,u)
    c=f(s,u)-A@s-B@u
    # END PART (b) #########################################################
    return A, B, c


def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters):
    """Solve the cart-pole swing-up problem via SCP.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
    s_goal : numpy.ndarray
        The goal state (1-D).
    N : int
        The time horizon of the LQR cost function.
    P : numpy.ndarray
```

```
        The terminal state cost matrix (2-D).
    Q : numpy.ndarray
        The state stage cost matrix (2-D).
    R : numpy.ndarray
        The control stage cost matrix (2-D).
    u_max : float
        The bound defining the control set `[-u_max, u_max]`.
    ρ : float
        Trust region radius.
    eps : float
        Termination threshold for SCP.
    max_iters : int
        Maximum number of SCP iterations.

    Returns
    -------
    s : numpy.ndarray
        A 2-D array where `s[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u : numpy.ndarray
        A 2-D array where `u[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    J : numpy.ndarray
        A 1-D array where `J[i]` is the SCP sub-problem cost after the i-th
        iteration, for `i = 0, 1, ..., (iteration when convergence occured)`
    """
    n = Q.shape[0]  # state dimension
    m = R.shape[0]  # control dimension

    # Initialize dynamically feasible nominal trajectories
    u = np.zeros((N, m))
    s = np.zeros((N + 1, n))
    s[0] = s0
    for k in range(N):
        s[k + 1] = fd(s[k], u[k])

    # Do SCP until convergence or maximum number of iterations is reached
    converged = False
    J = np.zeros(max_iters + 1)
    J[0] = np.inf
    for i in (prog_bar := tqdm(range(max_iters))):
        s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, ρ)
        dJ = np.abs(J[i + 1] - J[i])
        prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
        if dJ < eps:
            converged = True
            print("SCP converged after {} iterations.".format(i))
            break
    if not converged:
        raise RuntimeError("SCP did not converge!")
    J = J[1 : i + 1]
    return s, u, J


def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, ρ):
    """Solve a single SCP sub-problem for the cart-pole swing-up problem.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
    s_goal : numpy.ndarray
        The goal state (1-D).
    s_prev : numpy.ndarray
        The state trajectory around which the problem is convexified (2-D).
    u_prev : numpy.ndarray
        The control trajectory around which the problem is convexified (2-D).
    N : int
        The time horizon of the LQR cost function.
    P : numpy.ndarray
        The terminal state cost matrix (2-D).
```

```
    Q : numpy.ndarray
        The state stage cost matrix (2-D).
    R : numpy.ndarray
        The control stage cost matrix (2-D).
    u_max : float
        The bound defining the control set `[-u_max, u_max]`.
    ρ : float
        Trust region radius.

    Returns
    -------
    s : numpy.ndarray
        A 2-D array where `s[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u : numpy.ndarray
        A 2-D array where `u[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    J : float
        The SCP sub-problem cost.
    """
    A, B, c = affinize(f, s_prev[:-1], u_prev)
    A, B, c = np.array(A), np.array(B), np.array(c)
    n = Q.shape[0]
    m = R.shape[0]
    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    # PART (c) ############################################################
    # INSTRUCTIONS: Construct the convex SCP sub-problem.
    objective = 0.0
    constraints = []
    constraints += [s_cvx[0] == s0]
    constraints+=[cvx.norm(s_cvx[k]-s_prev[k],'inf')<=ρ for k in range(N+1)]
    constraints+=[cvx.norm(u_cvx[k]-u_prev[k],'inf')<=ρ for k in range(N)]
    constraints+=[cvx.norm(u_cvx[k], 'inf') <= u_max for k in range(N)]
    constraints += [s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k] for k in range(N)]
    objective += cvx.quad_form(s_cvx[N] - s_goal, P)
    objective += cvx.sum([cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R) for k in range(N)])
    # END PART (c) ########################################################

    prob = cvx.Problem(cvx.Minimize(objective), constraints)
    prob.solve()
    if prob.status != "optimal":
        raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
    s = s_cvx.value
    u = u_cvx.value
    J = prob.objective.value
    return s, u, J


def cartpole(s, u):
    """Compute the cart-pole state derivative."""
    mp = 1.0  # pendulum mass
    mc = 4.0  # cart mass
    L = 1.0  # pendulum length
    g = 9.81  # gravitational acceleration

    x, θ, dx, dθ = s
    sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = jnp.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


def discretize(f, dt):
    """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""
```

```python

    def integrator(s, u, dt=dt):
        k1 = dt * f(s, u)
        k2 = dt * f(s + k1 / 2, u)
        k3 = dt * f(s + k2 / 2, u)
        k4 = dt * f(s + k3, u)
        return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    return integrator


# Define constants
n = 4  # state dimension
m = 1  # control dimension
s_goal = np.array([0, np.pi, 0, 0])  # desired upright pendulum state
s0 = np.array([0, 0, 0, 0])  # initial downright pendulum state
dt = 0.1  # discrete time resolution
T = 10.0  # total simulation time
P = 1e3 * np.eye(n)  # terminal state cost matrix
Q = np.diag([1e-2, 1.0, 1e-3, 1e-3])  # state cost matrix
R = 1e-3 * np.eye(m)  # control cost matrix
ρ = 1.0  # trust region parameter
u_max = 8.0  # control effort bound
eps = 5e-1  # convergence tolerance
max_iters = 100  # maximum number of SCP iterations
animate = False  # flag for animation

# Initialize the discrete-time dynamics
fd = jax.jit(discretize(cartpole, dt))

# Solve the swing-up problem with SCP
t = np.arange(0.0, T + dt, dt)
N = t.size - 1
s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters)

# Simulate open-loop control
for k in range(N):
    s[k + 1] = fd(s[k], u[k])

# Plot state and control trajectories
fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
plt.subplots_adjust(wspace=0.45)
labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
labels_u = (r"$u(t)$",)
for i in range(n):
    ax[i].plot(t, s[:, i])
    ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
    ax[i].set_xlabel(r"$t$")
    ax[i].set_ylabel(labels_s[i])
for i in range(m):
    ax[n + i].plot(t[:-1], u[:, i])
    ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
    ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
    ax[n + i].set_xlabel(r"$t$")
    ax[n + i].set_ylabel(labels_u[i])
plt.savefig("cartpole_swingup_constrained.png", bbox_inches="tight")

# Plot cost history over SCP iterations
fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
ax.semilogy(J)
ax.set_xlabel(r"SCP iteration $i$")
ax.set_ylabel(r"SCP cost $J(\bar{x}^{(i)}, \bar{u}^{(i)})$")
plt.savefig("cartpole_swingup_constrained_cost.png", bbox_inches="tight")
plt.show()

# Animate the solution
if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup_constrained.mp4", writer="ffmpeg")
    plt.show()
```